

DEVELOPMENT DOCUMENT · VERSION 1.2

Solar Site Fitness & Capacity Engine

Business Cases · Functional Requirements · Developer Guide

Part I — Business Cases and Unit Economics

Part II — Functional Requirements

Part III — Technology Stack and Developer Guide

Companion to Product Specification v2.0

First published 7 August 2026 · Revised 24 August 2026 (v1.2)

INTERNAL — PRODUCT DESIGN

What changed in v1.2 Two additions on top of v1.1's rooftop-intelligence pipeline, both filling gaps flagged during rooftop-scope implementation planning: obstacle detection now actually affects usable area (previously only advisory notes), and shading — already present in the Solar API response but never extracted — now feeds both the generation estimate and the fitness score.

- **New pipeline stage:** obstacle detection folds into the existing GPT-4 Vision refinement call (no new crop, no new API call) and — uniquely among every v1.1/v1.2 addition — a high-confidence detection *auto-applies* to the site's exclusions, versioned and reversible exactly like any other boundary change — new §9.16 (`OBS`).
- **New scoring input:** per-roof-segment shading/sunshine data already returned by the Solar API is now extracted and threaded through as a generation-estimate derate and a fitness sub-score — new §9.17 (`SHADE`).

What changed in v1.1

This revision layered a rooftop/site-image intelligence pipeline and a Next.js frontend onto the v1.0 engine. The core decision in Part III — Python/FastAPI for the entire backend — is unchanged and, if anything, further reinforced: GPT-4 Vision, 3D reconstruction and ML scoring are all native Python workloads.

- **Frontend:** Next.js (React, TypeScript) replaces the Vite SPA — see §10.3.
- **New external dependency:** a Weather API, alongside the existing Google Maps and Google Solar API integrations.
- **New pipeline stage:** provider-derived boundaries are cropped and sent to GPT-4 Vision for inference-time refinement, then used to generate a 3D panoramic model — §9.11–9.12 (`VIS` , `VIZ`).
- **New scoring layer:** an ML suitability model trained on accumulated pipeline data, stored as additive metadata alongside the existing rules-based verdict — §9.13 (`ML`).
- **New caching layer:** a site already analysed at a given location — by anyone — is never re-processed; results are served from storage with zero repeat API calls — §9.14 (`CACHE`).
- **New intake field:** USN (DISCOM Unique Service Number) capture for billing-linked rooftop sites, from manual entry, bill OCR, or payment-proof OCR — §9.15 (`USN`).

PART I

Business Cases

Who buys this, why, and what it is worth to them

1. What Is Being Sold

The product answers two questions about any candidate site: can solar be installed, and at what capacity. Around that engine sit five distinct buyers with different triggers, cycles and price points. The engine is the same for all five; only the constraint pack and the packaging differ.

The unit of value is **a decision avoided or a decision improved**. A portfolio owner with 800 water bodies or 40,000 buildings cannot survey them all. What they buy is the ability to spend a fixed survey budget on the right subset.

2. Business Case A — State Agency, Floating Solar

Dimension	Detail
Buyer	State renewable energy nodal agency (NREDCAP, TEDA, MEDA, GEDA, KREDL and peers); occasionally the irrigation or water resources department.
Trigger	PM Surya Sarovar Yojana, approved 31 July 2026: ₹5,070 crore, 5,000 MW of floating solar with storage, implemented through SECI, running FY 2026-27 to FY 2030-31. Central assistance of ₹1 crore per MW on commissioning, and up to ₹50 lakh per project for pre-construction feasibility studies including bathymetry and hydrographic survey.
Pain	The state has several hundred to a few thousand reservoirs, tanks and industrial ponds. It has no inventory of which are viable, at what capacity, or in what order. Bathymetry is expensive and must be targeted. Without a candidate list, the state cannot even frame its allocation request.
What we deliver	A ranked candidate inventory: every water body in the state with minimum multi-year water spread, seasonal variability, indicative MW at the applicable coverage limit, evacuation proximity and headroom, ownership, and a fitness verdict with confidence.
Why it lands	The scheme has already priced feasibility work, so the budget objection does not arise. Our output is precisely the input to deciding where the ₹50 lakh per project should go.
Cycle	3–9 months. Tender or GEM route. Faster where framed as a study rather than a software purchase.
Indicative pricing	Statewide inventory engagement, plus per-water-body deep assessment for the shortlist, plus an annual platform subscription for refresh and tracking. See Section 7.

Why this is the entry case

Water body delineation is automated from free Sentinel-2 imagery. We can produce a credible statewide inventory *before* any customer engagement, at near-zero marginal cost, and walk in with the artefact rather than a proposal. No competitor has one, because the scheme is weeks old. Geo-based result caching (\$9.14) keeps this near-zero marginal cost even as coverage grows — a water body already analysed anywhere on the platform is never re-processed.

3. Business Case B — State Agency, Government Building Rooftops

Dimension	Detail
Buyer	State nodal agency; sometimes an individual department (Education, Health, Police) with its own energy budget.
Trigger	State rooftop targets under the clean energy policy; departmental opex reduction mandates; RESCO tendering that requires a defined building portfolio.
Pain	Tens of thousands of buildings, no roof or load data, survey capacity in the low hundreds per month. Surveys are spent as much on unsuitable buildings as on viable ones.
What we deliver	Roof geometry capture, usable area, capacity bound by net metering and consumption and transformer headroom, feasibility verdict, ranked survey queue, and reconciliation of field results against estimates — now including a GPT-4-Vision-refined rooftop crop and an auto-generated 3D panorama as part of the pre-feasibility artefact, so the agency sees the roof, not just a number.
Value logic	If survey conversion rises from roughly one in three to roughly two in three, the survey cost per converted site halves. On a programme running a few thousand surveys a year, that saving is an order of magnitude larger than the software cost.
Cycle	6–18 months, tender-driven. The NREDCAP engagement is the reference that shortens every subsequent one.
Constraint	Geometry capture is manual in India. At portfolio scale this is a services cost, not pure software. Price accordingly.

4. Business Case C — RESCO / Developer Portfolio

Dimension	Detail
Buyer	RESCO operators and IPPs bidding on multi-site tenders.
Trigger	A tender lands with 300–2,000 buildings and a 30-day bid window. The bidder must price a portfolio it has never visited.
Pain	Bid teams either over-price to cover unknown risk and lose, or under-price and discover unsuitable sites after award. Neither is recoverable.
What we deliver	Portfolio-level assessment inside the bid window: aggregate installable capacity, P50 and P90 generation, share of sites likely to fail survey, and a per-site risk flag — with a 3D panorama view per site available for buyer sign-off before the bid is submitted. Output feeds directly into the financial model.
Why they pay quickly	Commercial buyer, no tender to win the work, decision in days. A single avoided mis-bid on a portfolio dwarfs any plausible fee.
Cycle	1–4 weeks. The fastest revenue in the product.

5. Business Case D — DISCOM, PM Surya Ghar Targeting

Dimension	Detail
Buyer	Distribution utility; state programme cell.
Pain	Household rooftop targets are pursued through undifferentiated outreach. Some feeders have good roof stock and transformer headroom; others will not absorb the generation. Nobody knows which.
What we deliver	Feeder and DT-level aggregation: roof stock within the feeder polygon, aggregate installable capacity, transformer headroom, consumer mix, and a ranked targeting list for outreach and subsidy deployment.
Distinction	Crores of households make per-site assessment neither affordable nor necessary. The composite-site model in Specification v2.0 §8 handles this without new engine code.
Cycle	9–24 months. Large, slow, high-value. Not a first market.

6. Business Case E — Corporate / C&I Portfolio

Dimension	Detail
Buyer	Energy or sustainability head at a retail chain, bank, hotel group, hospital network or manufacturer with many sites.
Trigger	Board-level emissions commitment; opex reduction target; an RE100-style disclosure needing a defensible number.
What we deliver	Portfolio solar potential across all sites, phased by payback, with the open-access and captive rules of each state applied — plus an audit-ready basis for the disclosure.
Why it converts	Commercial procurement, annual subscription, expands site by site. Lowest friction of the five.
Cycle	2–8 weeks.

7. Pricing Model

Three components, mixed by segment. Per-assessment pricing aligns revenue with cost, because the dominant variable costs — Maps calls, compute, and operator time for manual geometry — all scale per site.

Component	Basis	Applies to
Platform subscription	Annual, tiered by portfolio size	All segments. Covers access, refresh, dashboards, support.
Per-assessment fee	Per site assessed, tiered by site type and depth	Volume driver. Floating commands a far higher unit price than rooftop, reflecting site value.
Engagement fee	Fixed-price study	Statewide inventories, RESCO bid support. Where procurement prefers a study to a licence.

Set unit prices from the value of the decision, not the cost of compute

A floating solar candidate is a multi-crore project; a wrong call on one water body costs far more than an entire state inventory. A rooftop assessment on a small government building is worth a few hundred rupees at most. Pricing the two identically because they run through the same engine would leave most of the value uncaptured on one side and price the product out on the other.

Before publishing a rate card, model gross margin per site type with realistic operator minutes for

manual geometry. Rooftop at portfolio scale is a services business with software leverage; floating is a software business. They should not carry the same margin assumption.

8. Sequencing the Market

Order	Segment	Reason to go here at this point
1	State agency — floating	Funded feasibility line, automated geometry, no incumbent inventory, scheme is weeks old. Build the artefact before selling.
2	RESCO portfolio	Fast commercial revenue while agency tenders run. Proves the product outside government.
3	State agency — rooftop	Requires the drawing productivity work first. NREDCAP as reference.
4	Corporate C&I	Subscription revenue, low friction, expands naturally.
5	DISCOM — PM Surya Ghar	Largest and slowest. Needs the composite-site model and references from 1–3.

PART II

Functional Requirements

The engine, stated as buildable requirements

9. Scope of This FRD

This covers the engine and the geometry capture surface — the parts that are the product. Registry, dashboards and survey workflow are packaging: specified in v1.0, optional per deployment, and omitted here except where the engine depends on them.

Requirement prefixes: **SITE** site model · **GEO** geometry providers · **AREA** usable area · **CON** constraints · **CAP** capacity resolver · **GEN** generation · **FIT** fitness scoring · **API** interface · **CAL** calibration · **CFG** configuration · **VIS** vision refinement · **VIZ** 3D visualization · **ML** ML suitability model · **CACHE** result cache · **USN** billing-identifier capture · **OBS** obstacle detection **NEW** · **SHADE** shading analysis **NEW**.

9.1 Site Model — SITE

ID	Requirement	Priority
SITE-01	Store a site with type discriminator, name, owner organisation, jurisdiction, centroid, boundary polygon and exclusion multipolygon.	Must
SITE-02	Validate type-specific attributes against a JSON Schema registered per site type; reject writes that fail. The schema for <code>ROOFTOP_RESIDENTIAL</code> / <code>ROOFTOP_CI</code> includes the USN field group (\$9.15); other site types omit it entirely rather than requiring-then-ignoring it.	Must
SITE-03	Support at minimum <code>ROOFTOP_GOVT</code> , <code>ROOFTOP_RESIDENTIAL</code> , <code>ROOFTOP_CI</code> , <code>FLOATING</code> at first release; <code>GROUND_MOUNT</code> , <code>CANAL_TOP</code> , <code>CARPORT</code> thereafter without engine change.	Must
SITE-04	Persist geometry source, imagery date, imagery quality and geometry confidence with every boundary.	Must
SITE-05	Version boundary changes rather than overwriting; retain full history with actor and timestamp.	Must
SITE-06	Support a composite site whose boundary is an aggregation of many member geometries (feeder-level use).	Should
SITE-07	Detect probable duplicates on spatial proximity and name similarity at creation and bulk import.	Should

9.2 Geometry Providers — GEO

ID	Requirement	Priority
GEO-01	Register providers against site types; resolve the applicable provider chain at runtime with a defined precedence order.	Must
GEO-02	<code>MANUAL_POLYGON</code> : operator draws boundary and exclusions on satellite imagery, edits vertices after closure, emits GeoJSON.	Must
GEO-03	<code>WATER_INDEX</code> : derive water spread from Sentinel-2 multi-temporal MNDWI; emit per-date polygons and a seasonal profile.	Must
GEO-04	<code>SOLAR_API</code> : retrieve building insights where covered via the Google Solar API; handle absent, BASE-tier and sparse responses without failure, recording outcome. Feeds the customer-selected geo-location flow: address/pin → Google Maps geocode → Solar API building insights → boundary handed to <code>VIS</code> (\$9.11) for refinement.	Should
GEO-05	<code>IMPORTED</code> : accept GeoJSON and shapefile upload with CRS detection and validation.	Must
GEO-06	<code>FIELD_MEASURED</code> : supersede any remote geometry on validated survey submission; retain the superseded version.	Must
GEO-07	Reject geometry that self-intersects, has fewer than three distinct vertices, lies beyond a configurable distance from the site centroid, or falls outside plausible area bounds for its type.	Must
GEO-08	Reject an exclusion polygon not contained within its boundary.	Must
GEO-09	Compute geometry confidence from source, imagery recency, vertex count and area plausibility.	Must

9.3 Usable Area — AREA

ID	Requirement	Priority
AREA-01	Compute boundary area in square metres using a method valid for geographic coordinates — geography cast or projected CRS. Never planar area on EPSG:4326.	Must
AREA-02	Compute total exclusion area with overlapping exclusions unioned, not summed.	Must
AREA-03	Apply a configurable edge setback as a negative buffer in a projected CRS.	Must
AREA-04	Compute usable area as boundary less setback less exclusions less type-specific deductions.	Must
AREA-05	Apply a utilisation factor by class where per-site precision is unavailable; record the factor and its version.	Must
AREA-06	Return zero usable area, not a negative value, where setback consumes the boundary.	Must

9.4 Constraints — CON

ID	Requirement	Priority
CON-01	Implement a constraint interface returning a capacity ceiling in kWp, a reason, a confidence delta and a status.	Must
CON-02	Group constraints into versioned packs scoped by site type and jurisdiction; resolve the applicable pack at assessment time.	Must
CON-03	Distinguish ceilings from gates. Gates are binary, never enter the minimum, and modify the verdict.	Must
CON-04	Return INSUFFICIENT_DATA from a constraint that cannot be evaluated — never a zero or default ceiling.	Must
CON-05	Ship the rooftop pack: usable area, net-metering cap, consumption offset, transformer headroom, structural gate, subsidy tier cap. For <code>ROOFTOP_RESIDENTIAL</code> / <code>ROOFTOP_CI</code> , the captured USN (\$9.15) is an input value to the net-metering/subsidy-tier lookup — it does not introduce a new constraint of its own.	Must
CON-06	Ship the floating pack: minimum water spread, coverage limit, anchoring depth window, drawdown tolerance, storage co-location, reserved-use exclusions, ownership gate.	Must
CON-07	Ship universal constraints: usable area, evacuation headroom, minimum viable size.	Must
CON-08	Permit a jurisdiction pack to override a national default without code change.	Must
CON-09	Evaluate constraints independently and in any order; no constraint may depend on another's output.	Must

9.5 Capacity Resolver — CAP

ID	Requirement	Priority
CAP-01	Compute recommended capacity as the minimum of all evaluable ceilings.	Must
CAP-02	Identify and return the binding constraint.	Must
CAP-03	Return maximum technical capacity — the minimum of physical ceilings only, excluding regulatory and commercial ones.	Must
CAP-04	Return headroom: the gap between the binding ceiling and the next lowest.	Must
CAP-05	Return the full ceiling list with each constraint's value and status, not merely the minimum.	Must
CAP-06	State the unit basis (DC) explicitly on every capacity figure.	Must

9.6 Generation — GEN

ID	Requirement	Priority
GEN-01	Fast estimate: capacity × location-specific specific yield × performance ratio.	Must
GEN-02	Resolve specific yield by district or grid cell from a reference dataset, refined by current Weather API conditions (irradiance, temperature, cloud cover) where available — never a single national or state constant.	Must
GEN-03	Apply a site-type performance adjustment (floating arrays run cooler than rooftop; the uplift is modest and must be configurable, not assumed).	Must
GEN-04	Detailed estimate using plane-of-array irradiance, temperature and inverter losses, sourced from the Weather API where enabled.	Should
GEN-05	Record the estimation method on every result so fast and detailed figures are never conflated.	Must
GEN-06	Produce P50 and P90 figures for portfolio aggregation.	Should

9.7 Fitness Scoring — FIT

ID	Requirement	Priority
FIT-01	Compute a fitness score from weighted components defined in the scoring profile for the site type.	Must
FIT-02	Assign a verdict: SUITABLE, SUITABLE_SUBJECT_TO_SURVEY, CONDITIONAL, INSUFFICIENT_DATA, NOT_SUITABLE.	Must
FIT-03	Return INSUFFICIENT_DATA where any mandatory input is absent, in precedence over any computed score.	Must
FIT-04	Compute confidence from geometry source, imagery recency, constraint data completeness, gate resolution and model calibration state.	Must
FIT-05	Emit a human-readable reason list naming the binding constraint and any estimated or missing inputs.	Must
FIT-06	Never return a verdict or capacity without confidence and binding constraint. This is the sole authoritative, reproducible verdict — the ML suitability score (§9.13) is stored separately and never substitutes for it.	Must
FIT-07	Attach the standard pre-feasibility limitations statement to every result.	Must

9.8 Interface — API

ID	Requirement	Priority
API-01	Single assessment response shape for all site types, per Specification v2.0 §6. The response includes, where present, the <code>vision_refinement</code> , <code>panorama_url</code> , <code>ml_suitability_score</code> and <code>cache_hit</code> fields as optional, additive members — never required, never displacing the core shape.	Must
API-02	Assess a single site synchronously where inputs are already present, or where a cache hit (§9.14) resolves the full pipeline instantly.	Must
API-03	Submit batch assessment asynchronously, returning a job id with pollable status and per-site results.	Must
API-04	Return engine version and constraint pack version on every response.	Must
API-05	Version the API path; never break an existing response shape.	Must
API-06	Authenticate by API key scoped to a tenant; enforce per-tenant rate limits and quotas.	Must
API-07	Bulk import sites from CSV, GeoJSON or shapefile with per-row validation and an error report.	Must
API-08	Export assessments as CSV, GeoJSON and PDF report.	Should
API-09	Emit a webhook on assessment completion.	Should

9.9 Calibration — CAL

ID	Requirement	Priority
CAL-01	On field survey submission, compute variance between remote and measured usable area and store the labelled pair with site type, region, geometry source and class.	Must
CAL-02	Flag records where variance exceeds a configurable threshold and mark the remote estimate superseded.	Must
CAL-03	Recompute utilisation factors per class once the sample count for that class exceeds a threshold; do not apply silently — surface the proposed change for approval.	Must
CAL-04	Report variance distribution by site type, region, geometry source and class.	Should
CAL-05	Feed calibration state into the confidence model for future assessments of the same class, and into the ML model's offline retraining set (§9.13) as labelled outcomes once available.	Must

9.10 Configuration — CFG

ID	Requirement	Priority
CFG-01	Hold every coefficient in a versioned parameter pack. No numeric constant in application logic. The lat/long rounding precision used by the result cache (§9.14) is a parameter-pack value, not a hard-coded constant.	Must
CFG-02	Record the parameter pack version and constraint pack version against every stored result.	Must
CFG-03	Reproduce any historical assessment exactly from stored inputs and recorded versions.	Must
CFG-04	Support bulk recomputation across a jurisdiction after a pack change, retaining prior results.	Must
CFG-05	Scope packs by tenant so a customer may hold private overrides.	Should

9.11 Vision Refinement — VIS

After any **GEO** provider produces a boundary — Solar API for rooftop, water-index for floating, imported/manual for either — the source imagery is cropped to that boundary and passed to a vision-capable LLM (GPT-4 Vision) for a second-pass, inference-time refinement. This is explicitly **not** model fine-tuning: no weights are trained, no imagery is retained as training data, and no vendor model is customised. The call corrects or flags boundary issues and notes visible obstructions the upstream provider may have missed.

ID	Requirement	Priority
VIS-01	Crop the source imagery to the provider-derived boundary before any downstream use — never send an uncropped tile.	Must
VIS-02	Send the crop to the configured vision-LLM for a refinement pass; parse the response into a structured annotation (corrected-boundary suggestion, obstruction notes, confidence).	Must
VIS-03	Store the refinement as an annotation on the analysis record — never overwrite the provider's original geometry. A human or the calibration path (§9.9) decides whether to promote a refinement into the boundary of record.	Must
VIS-04	On vision-call failure or low-confidence response, mark the annotation <code>insufficient_data</code> and proceed with the unrefined boundary — never block the pipeline on this step.	Must
VIS-05	Run behind the same async worker pattern as the water-index provider (§9.2) — never in the request path.	Must
VIS-06	Treat the cropped imagery sent to a third-party vision API under the same imagery-licensing discipline as §17's Google-imagery restriction: no bulk retention, no use as training data, storage of the crop itself gated on licence review — flagged as an open legal item, not assumed clear.	Must

9.12 3D Visualization — VIZ

From the confirmed boundary (post- `VIS` where available) plus elevation/imagery data, the engine builds a 3D panoramic model of the site — a rooftop, a floating array's water surface, or a ground-mount plot — for visual review by the buyer alongside the numeric verdict.

ID	Requirement	Priority
VIZ-01	Generate a 3D mesh/panorama from the site boundary and available elevation data (Solar API data layers where present, otherwise a configured DEM source).	Must
VIZ-02	Store the generated artifact in object storage; persist only a reference URL plus generation parameters and a version identifier on the site record — geometry itself is never re-derived from the artifact.	Must
VIZ-03	Where elevation data or imagery quality is insufficient to produce a usable model, return an explicit <code>not_generated</code> status with reason — never fabricate a plausible-looking mesh.	Must
VIZ-04	Regenerate only on an explicit boundary change (a new <code>SITE-05</code> version) or an admin-triggered refresh — never on every page view.	Must
VIZ-05	Run as an async worker task, chained after <code>VIS</code> completes (or is skipped) — never in the request path.	Must

9.13 ML Suitability Model — ML

An ML model, trained offline on accumulated `GEO` / `AREA` / `VIS` /weather feature data with `FIT` verdicts and, once available, `CAL` field-survey outcomes as labels, produces a supplementary suitability score. This is **additive metadata**: the deterministic, versioned `FIT` verdict (§9.7) remains the sole authoritative and reproducible output per `FIT-06` and `CFG-03`. The ML score exists to be compared against the rules engine over time, not to replace it.

ID	Requirement	Priority
ML-01	Compute and store <code>ml_suitability_score</code> alongside, never in place of, the <code>FIT</code> verdict.	Must
ML-02	Record the model version against every stored score, following the same versioning discipline as <code>CFG-01/02</code> — a historical ML score must be traceable to the model that produced it, even though it sits outside the reproducibility guarantee that covers the rules engine.	Must
ML-03	Retrain only as an offline/scheduled job; never trigger training or weight updates from the request path.	Must
ML-04	Surface a newly trained model version for approval before it becomes the active scorer — mirrors <code>CAL-03</code> 's "propose, don't apply silently" pattern.	Must
ML-05	Where mandatory upstream inputs are missing, return <code>insufficient_data</code> for the ML score rather than a fabricated number — same discipline as <code>FIT-03</code> .	Must

9.14 Result Cache — CACHE

The full pipeline above — geometry provider, vision refinement, weather lookup, 3D generation, ML scoring — runs once per distinct location. A second selection of the same building or water body, by the same tenant or a different one, is served entirely from stored results: zero external API calls, zero recompute.

ID	Requirement	Priority
CACHE-01	Key cache lookups on latitude/longitude rounded to a parameter-pack-defined precision (<code>CFG-01</code>) — not on a site id, tenant id, or provider-specific building identifier, so the cache is shared across tenants and site records.	Must
CACHE-02	On site creation or geo-selection, look up the cache before invoking any <code>GEO</code> , <code>VIS</code> , weather, <code>VIZ</code> or <code>ML</code> step; on a hit, attach the cached results to the new site record with no external calls made.	Must
CACHE-03	Apply uniformly across site types (rooftop, floating, future ground-mount) — one cache table, one key scheme, independent of the <code>SITE-03</code> type discriminator.	Must
CACHE-04	Reuse is unconditional by default — no automatic expiry — but an explicit, admin-triggered "force refresh" path must exist for a known-stale entry. The pipeline never silently re-runs on its own schedule.	Must
CACHE-05	Record every cache hit as such (<code>reused_from_analysis_id</code> plus original and reuse timestamps) — a cached result must never be indistinguishable from a freshly computed one in the stored record or the API response.	Must

9.15 USN Capture — USN

Scoped to `ROOFTOP_RESIDENTIAL` and `ROOFTOP_CI` only — the site types tied to an individual DISCOM billing connection. The Unique Service Number (USN) is the utility-assigned connection identifier printed on the electricity bill, and is an input to the rooftop constraint pack's net-metering and subsidy-tier lookups (`CON-05`).

ID	Requirement	Priority
USN-01	Accept USN via manual text entry as the baseline path, available for every billing-linked rooftop site.	Must
USN-02	Extract USN via OCR from an uploaded electricity bill; surface the extracted value for the customer/operator to confirm or correct before it is stored.	Must
USN-03	Extract USN via OCR from a payment-proof or transaction screenshot (or any other document where USN is printed), as an alternative to a full bill upload; same confirm-before-store discipline as USN-02.	Must
USN-04	Record a single <code>usn</code> field with a <code>usn_source</code> tag (<code>manual</code> / <code>bill_ocr</code> / <code>payment_proof_ocr</code>) — the three input paths converge on one value, never three separate fields.	Must
USN-05	Omit the USN field group entirely from the JSON Schema (<code>SITE-02</code>) for any non-billing-linked site type — never present it as an ignored or optional-but-visible field on floating, government, ground-mount or composite sites.	Must
USN-06	Treat any bill or payment-proof image uploaded solely for USN OCR as sensitive personal/billing data: encrypted at rest, retained only as long as needed to support the extraction and any dispute/audit window, and never used as training data for the vision or ML models.	Must

9.16 Obstacle Detection — OBS NEW

The gap this closes: prior to v1.2, an obstacle spotted by `VIS-02`'s vision-LLM call surfaced only as unstructured, advisory-only `obstruction_notes` free text — never applied to usable area (`VIS-03` explicitly forbids auto-applying vision output). Obstacle detection folds into the **same** vision-LLM call as `VIS-02` — the same crop, the same request, never a second image or a second call — with the structured-output schema extended to return typed obstacles alongside the existing corrected-boundary suggestion. Unlike everything else in §9.11–9.15, a confident detection here **auto-applies** to a customer-visible number: it's the one deliberate exception to this document's advisory-only discipline, and it's held to a stricter audit standard as a result — confidence-gated, versioned exactly like any other boundary change, and independently reversible.

ID	Requirement	Priority
OBS-01	Fold obstacle detection into the same vision-LLM call as <code>VIS-02</code> — reuse <code>VIS-01</code> 's crop, never re-crop or re-send imagery separately. Extend the structured-output schema to also return <code>obstacles: list[{type, bounding_polygon, confidence}]</code> alongside <code>VIS-02</code> 's existing corrected-boundary suggestion.	Must
OBS-02	Support obstacle types at minimum: <code>water_tank</code> , <code>hvac_unit</code> , <code>chimney</code> , <code>existing_solar_panel</code> , <code>vent</code> , <code>antenna</code> , <code>other</code> .	Must
OBS-03	Validate every detected <code>bounding_polygon</code> against <code>GEO-07</code> / <code>GEO-08</code> 's self-intersection, vertex-count, and containment rules before any further processing — an invalid polygon is dropped with a recorded reason, never forced through.	Must
OBS-04	Obstacles at or above a configurable <code>auto_apply_confidence_threshold</code> (a <code>CFG-01</code> parameter-pack value, never hard-coded) are unioned into the site's exclusions and trigger a new boundary version via <code>SITE-05</code> , with <code>source=obstacle_detection</code> recorded on that version — fully auditable, never a silent change. <code>AREA-04</code> usable area is then recomputed automatically against the new exclusions.	Must
OBS-05	Obstacles below the threshold are stored as advisory-only annotations, same discipline as <code>VIS-03</code> — visible for admin review and manual promotion, never auto-applied.	Must
OBS-06	Because auto-apply changes a customer-visible number without a human step, every auto-applied obstacle must be independently reversible: an admin rejecting one supersedes that exclusion (retaining history per <code>SITE-05</code>) and triggers <code>AREA-04</code> recomputation back toward the prior value.	Must
OBS-07	Run behind the same async worker pattern as <code>VIS-05</code> — never in the request path.	Must
OBS-08	Inherit <code>VIS-06</code> 's imagery-licensing discipline in full — same crop, same call, no additional licensing surface since no new imagery leaves the system.	Must
OBS-09	Once field-survey ground truth for obstacle presence/absence exists, report detection accuracy via the <code>CAL</code> calibration loop, analogous to <code>CAL-01..05</code> 's geometry-variance tracking — feeds future confidence-threshold and model-version decisions.	Should

9.17 Shading Analysis — SHADE NEW

The gap this closes: the Google Solar API's Building Insights response already carries per-roof-segment sunshine/shading data (sunshine-hour quantiles that inherently account for shading from surrounding buildings and trees) — but nothing prior to v1.2 extracted or used it. No new external

call, no new imagery: this is purely a matter of wiring already-fetched data through to the two places it belongs, generation and fitness.

ID	Requirement	Priority
SHADE-01	Extract per-roof-segment sunshine/shading data already present in the <code>GEO-04</code> Solar API response — no new external call, no new imagery. Where the boundary comes from a non-Solar-API provider (<code>MANUAL_POLYGON</code> / <code>IMPORTED</code> / <code>FIELD_MEASURED</code>), shading data is absent and SHADE-04's insufficient-data path applies.	Must
SHADE-02	Store a normalised <code>shading_score</code> (0..1) per site, derived from the extracted sunshine data relative to the unobstructed regional maximum (the same reference dataset <code>GEN-02</code> already uses for specific yield) — record the derivation method, never an unexplained raw number.	Must
SHADE-03	Apply the shading score as a performance derate in the generation estimate — <code>GEN-01</code> 's performance ratio is adjusted downward proportionally to shading, and <code>GEN-05</code> 's estimation-method record notes that a shading derate was applied.	Must
SHADE-04	Feed the shading score into <code>FIT-01</code> as one of the weighted scoring components. Where shading data is unavailable (non-Solar-API geometry source), return <code>INSUFFICIENT_DATA</code> for the shading sub-component specifically — never assume zero shading, never assume full shading.	Must
SHADE-05	Where a future custom shadow-casting model becomes justified (nearby-building/tree geometry plus sun-position simulation), it must produce the same <code>shading_score</code> shape as SHADE-02 so <code>FIT</code> / <code>GEN</code> need no interface changes — an implementation swap, not an interface swap.	Should

PART III

Technology Stack and Developer Guide

The stack decision, the libraries, and where to start

10. Python or Node — the Decision

This is the only backend-language question that materially affects the outcome, so it is worth settling on evidence rather than preference. The v1.1 additions — GPT-4 Vision calls, 3D mesh/panorama generation, and ML model training/serving — are, if anything, further evidence for the same

conclusion: they are scientific-computing and ML workloads with mature, standard Python tooling and no serious Node equivalent.

10.1 What the backend actually has to do

Strip away the CRUD and the backend is: read Sentinel-2 rasters, compute band indices over time series, threshold and vectorise, run polygon algebra in projected coordinate systems, evaluate constraint functions, model PV generation, refine geometry via a vision model, reconstruct 3D surfaces, and serve a trained scoring model. That is a scientific computing and ML workload wearing a web API as a hat.

Capability	Python	Node
Vector geometry	Shapely (GEOS bindings) — complete, correct, battle-tested	Turf.js — good for browser work, weaker on projected operations and large batches
Projections	pyproj (PROJ bindings) — full EPSG registry	proj4js — partial, and a common source of subtle error
Raster I/O	Rasterio, GDAL — the reference implementation	No serious equivalent. This alone decides it.
Sentinel-2 / STAC	pystac-client, odc-stac, stackstac, xarray	Nothing comparable
Zonal statistics	rasterstats, xarray	None
PV modelling	pvlb — validated, maintained, standard	None
3D mesh / point-cloud NEW	trimesh, Open3D — standard for mesh construction from boundary + elevation	No maintained equivalent; would mean shelling out to Python anyway
ML training/serving NEW	scikit-learn, XGBoost — standard, same ecosystem as the geospatial stack	Would require a separate Python service regardless
Tabular / numeric	GeoPandas, NumPy, pandas	Ad hoc
Web API	FastAPI — fast, typed, OpenAPI out of the box	Express / NestJS — equally capable
Async I/O concurrency	Good with asyncio; adequate here	Slightly better for I/O-heavy fan-out
Type sharing with frontend	Generate TS types from OpenAPI	Native TS end to end

Decision: Python for the entire backend. Next.js (React, TypeScript) for the frontend.

The raster, geospatial, vision-refinement, 3D-reconstruction and ML rows are not close, and they are where the product's differentiation lives. Water body delineation from Sentinel-2, GPT-4 Vision crop refinement, 3D panorama generation and suitability-model serving are not implementable in Node without shelling out to Python — at which point Python is already in the stack and the only question is how many languages you are maintaining.

Node's advantages here are real but small: marginally better I/O fan-out, and native type sharing with React. Neither outweighs having to reimplement or subprocess the core — and neither depends on which React framework sits on top, which is why the frontend framework choice (Vite SPA vs. Next.js) is decoupled from this decision.

10.2 The tempting hybrid, and why to avoid it

A common instinct is Node for the API and Python for the analysis service. It is defensible at large team size. For a small team building a first product it costs more than it returns:

- Two runtimes, two dependency managers, two CI pipelines, two deployment paths, two sets of security patching.
- Domain models defined twice — site, constraint, ceiling, assessment — and drifting apart at the first schema change.
- Every assessment crosses a network boundary with serialisation on both sides, for no functional gain.
- A bug in a capacity figure now requires reasoning across two codebases.

Choose Node instead only if the team is genuinely Node-only and hiring Python is not viable, or an existing Node platform must host this as a module. In that case run Python as a pure analysis worker behind a queue, keep every domain decision in it, and let Node do nothing but transport and auth.

10.3 Recommended stack

Layer	Choice	Note
API framework	FastAPI + Pydantic v2	OpenAPI schema generated automatically; feeds frontend type generation.
ORM	SQLAlchemy 2.0 + GeoAlchemy2	Geometry columns as first-class types.
Migrations	Alembic	—
Database	PostgreSQL 16 + PostGIS 3.4	Non-negotiable. Spatial indexing and projection support.
Jobs	Celery + Redis	Batch assessment, raster work, vision-refinement calls and 3D generation. Never in the request path.
Geometry	Shapely 2.x, pyproj	Shapely 2 is vectorised and materially faster than 1.x.
Raster	Rasterio, rioarray, xarray	—
Imagery access	pystac-client + odc-stac	Sentinel-2 L2A via a public STAC endpoint. See §13.
PV modelling	pvlb	Phase 2. Do not hand-roll irradiance.
Weather API client NEW	Typed client over a chosen provider (e.g. OpenWeatherMap / Visual Crossing / IMD)	Provider selection is an open decision — flagged here, not hard-picked. Feeds <code>GEN-02/04</code> and the <code>ML</code> feature set.
Vision refinement NEW	<code>openai</code> Python SDK (GPT-4 Vision)	Called from <code>providers/vision.py</code> , async, behind the same Celery worker pattern as the water-index provider. Never in the request path.
3D generation NEW	trimesh / Open3D (mesh) + pyrender or headless matplotlib (render)	Mesh built from boundary + elevation data; rendering step produces the panoramic artifact. New <code>engine/panorama.py</code> .
ML model serving NEW	scikit-learn / XGBoost, trained offline	Served from <code>engine/ml_score.py</code> ; model artifacts versioned in object storage alongside parameter packs.
Result cache NEW	Redis (hot path) + a durable Postgres table <code>site_analysis_cache</code>	Redis alone isn't the source of truth — reuse must be indefinite and auditable (<code>CACHE-04/05</code>), so the durable half lives in Postgres.

Frontend	Next.js (React, TypeScript)	Replaces the Vite SPA. Server-side rendering reduces first-page-load time for portfolio/dashboard views — the same rationale used by the companion GoHarit product's frontend decision.
Map	@vis.gl/react-google-maps	Declarative, maintained. Unchanged by the Next.js swap.
Drawing	terra-draw + Google Maps adapter	Google's Drawing Library was removed in Maps JS v3.65.3b (June 2026). Terra Draw is the named alternative.
Client geometry	@turf/turf	Live area feedback only. Server remains authoritative.
Data overlays	deck.gl	Maps Heatmap Layer also removed; deck.gl replaces it and scales far better.
3D/panorama viewer NEW	@react-three/fiber (Three.js) or a panorama-image viewer, per artifact type	Renders the VIZ artifact URL client-side; the artifact itself is generated server-side, never in the browser.
Server state	TanStack Query	—
Type sharing	openapi-typescript	Generates TS types from FastAPI's schema — works identically against Next.js as it did against the Vite SPA.

11. Repository Structure

```
solar-fitness/
├── apps/
│   ├── web/                                # Next.js (React + TypeScript)
│   │   ├── app/                            # Next.js routes (App Router)
│   │   └── src/
│   │       ├── features/
│   │       │   ├── sites/                  # list, detail, import
│   │       │   ├── map/
│   │       │   │   ├── components/         # MapView, SiteMarker, BoundaryPolygon,
│   │       │   │   │                       # ExclusionPolygon, LayoutOverlay
│   │       │   │   └── drawing/            # Terra Draw integration
│   │       │   ├── assessment/             # result view, ceilings, reasons
│   │       │   └── panorama/              # NEW – 3D/panorama artifact viewer
│   │       └── portfolio/                 # aggregation, saturation, export
│   └── lib/api/                           # generated from OpenAPI
├── api/                                   # FastAPI – the whole backend
│   └── src/solarfit/
│       ├── main.py
│       ├── routers/                        # sites, assessments, imports, config
│       ├── domain/
│       │   ├── site.py                    # entity + type registry
│       │   ├── constraint.py              # Constraint protocol, Ceiling, Gate
│       │   └── assessment.py              # result aggregate
│       ├── engine/
│       │   ├── area.py                    # AREA-01..06
│       │   ├── resolver.py                # CAP-01..06 (the MIN)
│       │   ├── generation.py              # GEN-01..06
│       │   ├── fitness.py                 # FIT-01..07
│       │   ├── panorama.py                # NEW – VIZ-01..05 (trimesh/Open3D + pyrender)
│       │   └── ml_score.py                # NEW – ML-01..05 (offline-trained model serving)
│       ├── providers/                     # geometry + enrichment providers
│       │   ├── base.py
│       │   ├── manual.py
│       │   ├── water_index.py             # Sentinel-2 MNDWI
│       │   ├── solar_api.py
│       │   ├── imported.py
│       │   ├── vision.py                  # NEW – VIS-01..06 (GPT-4 Vision refinement)
│       │   ├── weather.py                 # NEW – Weather API client
│       │   └── usn_ocr.py                 # NEW – USN-02/03 (bill / payment-proof OCR)
│       ├── packs/                         # constraint packs
│       │   ├── universal.py
│       │   ├── rooftop.py
│       │   ├── floating.py
│       │   └── jurisdictions/             # in_ap.py, in_tn.py, ...
│       ├── repositories/                  # SQLAlchemy
│       └── sites.py
```

```
| | | └─ analysis_cache.py # NEW - CACHE-01..05
| | └─ workers/           # Celery tasks
| └─ tests/
└─ packages/config-packs/  # versioned YAML parameter packs
└─ db/migrations/         # Alembic
└─ infra/
```

12. Core Interfaces

```
from typing import Protocol, Literal
from pydantic import BaseModel

class Ceiling(BaseModel):
    constraint: str
    ceiling_kwp: float | None          # None = could not evaluate
    reason: str
    confidence_delta: float = 0.0
    kind: Literal["physical", "regulatory", "commercial"] = "physical"
    status: Literal["ok", "estimated", "insufficient_data", "not_applicable"]

class Gate(BaseModel):
    gate: str
    status: Literal["PASS", "FAIL", "PENDING"]
    detail: str

class Constraint(Protocol):
    id: str
    applies_to: list[str]             # site types
    jurisdiction: str | None
    def evaluate(self, site, usable_area_m2, params) -> Ceiling | Gate: ...

# The resolver – CAP-01..06. Identical for every site type.
def resolve_capacity(ceilings: list[Ceiling]) -> CapacityResult:
    usable = [c for c in ceilings if c.ceiling_kwp is not None]
    if not usable:
        return CapacityResult(recommended_kwp=None, status="INSUFFICIENT_DATA")

    binding = min(usable, key=lambda c: c.ceiling_kwp)
    physical = [c for c in usable if c.kind == "physical"]
    ordered = sorted(usable, key=lambda c: c.ceiling_kwp)

    return CapacityResult(
        recommended_kwp = binding.ceiling_kwp,
        max_technical_kwp = min(c.ceiling_kwp for c in physical) if physical else None,
        binding_constraint = binding.constraint,
        headroom_kwp = (ordered[1].ceiling_kwp - binding.ceiling_kwp)
                        if len(ordered) > 1 else 0.0,
        ceilings = ceilings,
        unit_basis = "DC",
    )
```

Note how little there is. The resolver is roughly fifteen lines and never changes. Every site type, every programme and every jurisdiction is expressed in the constraint list handed to it. If that function starts acquiring conditionals on site type, the abstraction has been breached.

12.1 Cache lookup-or-compute — CACHE-01/02 NEW

Same discipline as the resolver above: small, boring, and it is the one function every geo-selection flow calls before anything expensive happens.

```
# Rounding precision comes from the parameter pack (CFG-01), never hard-coded here.
def get_or_create_analysis(lat: float, lng: float, site_type: str, params) -> AnalysisResult:
    key = round_latlng(lat, lng, precision=params.cache_precision)

    cached = analysis_cache_repo.find_by_key(key)
    if cached is not None:
        return cached.as_result(cache_hit=True, reused_from_analysis_id=cached.id)

    # No entry: run the full pipeline once, store it, and key it for reuse
    # by any tenant/site that selects this same location in future.
    boundary = run_geometry_providers(lat, lng, site_type, params)      # GEO
    refinement = run_vision_refinement(boundary, params)                # VIS
    weather = fetch_weather(lat, lng, params)                          # weather.py
    panorama = generate_panorama(boundary, weather, params)             # VIZ
    ml_score = score_with_ml_model(boundary, refinement, weather, params) # ML

    result = analysis_cache_repo.create(
        key=key, boundary=boundary, refinement=refinement,
        weather=weather, panorama=panorama, ml_score=ml_score,
    )
    return result.as_result(cache_hit=False)
```

13. Sentinel-2 Water Body Pipeline

The highest-value component and the one with least prior art in this domain. Free imagery, no licensing constraint, and no competitor has it. Unchanged in v1.1 — its output boundary is simply what now also flows into the `VIS` / `VIZ` / `ML` / `CACHE` stages described in Part II, on the same terms as a rooftop boundary.

13.1 Access

Sentinel-2 L2A is published as cloud-optimised GeoTIFFs with STAC catalogues on public cloud storage. Query with `pystac-client` and load lazily with `odc-stac` — only the bands and the spatial window needed are ever fetched.

```

import pystac_client, odc.stac, numpy as np

catalog = pystac_client.Client.open(STAC_ENDPOINT)

items = catalog.search(
    collections=["sentinel-2-l2a"],
    intersects=water_body_geojson,
    datetime="2021-06-01/2026-05-31",
    query={"eo:cloud_cover": {"lt": 20}},
).item_collection()

# B03 green (10 m) · B11 SWIR1 (20 m) · SCL scene classification (20 m)
ds = odc.stac.load(
    items,
    bands=["B03", "B11", "SCL"],
    resolution=10,          # B11 resampled up to match B03
    groupby="solar_day",
    chunks={},              # dask-backed; nothing loaded yet
    geopolygon=water_body_geojson,
)

```

13.2 Index and mask

```

# MNDWI, not NDWI. NDWI uses NIR and confuses built-up surfaces with water;
# MNDWI uses SWIR1 and separates them cleanly – which matters for tanks
# adjoining villages, i.e. most Indian irrigation tanks.
mndwi = (ds.B03 - ds.B11) / (ds.B03 + ds.B11)

# SCL classes: 3 cloud shadow, 8 cloud medium prob, 9 cloud high prob,
# 10 thin cirrus. Mask them before thresholding.
valid = ~ds.SCL.isin([3, 8, 9, 10])
water = (mndwi > MNDWI_THRESHOLD).where(valid)

```

The threshold is not a constant

MNDWI > 0 is the textbook value and it is a starting point, not an answer. Turbid water, shallow margins, dense aquatic vegetation and seasonal silt all shift it. Fix the threshold globally and tank areas will be systematically wrong in ways nobody notices.

Use Otsu thresholding per scene over the site window, which adapts to local conditions, and hold the fallback constant in the parameter pack. Record the threshold actually applied against each derived polygon — it is an input to geometry confidence, and it is the first thing to examine when a field survey disagrees with the estimate.

13.3 Seasonal profile

```

from rasterio.features import shapes

per_date = {}
for t in water.time:
    mask = water.sel(time=t).fillna(0).astype("uint8").values
    polys = [shape(g) for g, v in shapes(mask, transform=ds.odc.transform) if v == 1]
    per_date[str(t.values)[:10]] = unary_union(polys)

areas = {d: area_m2(p) for d, p in per_date.items()}

profile = {
    "max_spread_m2": max(areas.values()),
    "mean_spread_m2": statistics.mean(areas.values()),
    "min_spread_m2": min(areas.values()),          # ← governs FSPV capacity
    "cv": statistics.pstdev(areas.values()) / statistics.mean(areas.values()),
    "dry_months": months_at_minimum(areas),
    "scene_count": len(areas),
    "date_range": (min(areas), max(areas)),
}

# The site boundary is the MINIMUM spread polygon, not the mean or maximum.
boundary = per_date[min(areas, key=areas.get)]

```

Minimum spread, and why it is the whole point

A floating array must stay afloat and moored in the driest month of a dry year. Sizing to mean spread produces arrays that ground out — the commonest failure mode in early FSPV. Using the multi-year minimum is both the correct engineering choice and one that survives review.

The coefficient of variation is separately valuable and nobody else surfaces it. Two reservoirs with identical minimum area are very different propositions if one fluctuates twice as much, because the mooring design and its cost differ. Expose it as a ranking dimension.

13.4 Practical notes

- Require a minimum scene count before trusting a profile — a handful of cloud-free scenes across five years is not a seasonal profile. Set the floor in configuration and degrade confidence below it.
- Compute once per water body and cache — now the same durable cache described in §9.14/§12.1, not a bespoke per-provider cache. Refresh only on explicit force-refresh, not per request.
- Dask-backed lazy loading means memory scales with the site window, not the tile. A statewide run is embarrassingly parallel across bodies.
- Sentinel-2 at 10 m will not resolve small tanks reliably. Set a minimum area threshold below which the provider declines and defers to manual capture.

14. PostGIS Rules

The bug that will otherwise ship

`ST_Area()` on a `geometry(Polygon, 4326)` returns square degrees. No error, no warning, a plausible-looking number wrong by a factor of order 10^{10} that varies with latitude. Every downstream capacity, generation, investment and savings figure inherits the error.

```

-- Area: correct anywhere on earth
SELECT ST_Area(boundary::geography) AS area_m2 FROM sites;

-- Buffer in metres: requires a projected CRS.
-- ST_Buffer on 4326 reads the distance as DEGREES.
SELECT ST_Transform(ST_Buffer(ST_Transform(boundary, 32644), -0.5), 4326)
FROM sites WHERE id = :id;

-- Usable area in one statement
WITH p AS (
    SELECT ST_Transform(boundary, 32644) AS b,
           ST_Transform(exclusions, 32644) AS x
    FROM sites WHERE id = :id
)
SELECT ST_Area(ST_Difference(ST_Buffer(b, -:setback),
                             COALESCE(x, 'POLYGON EMPTY')))) AS usable_m2
FROM p;

-- Nearest substation with headroom – the evacuation ceiling
SELECT s.id, s.name, s.spare_capacity_mw,
       ST_Distance(site.centroid::geography, s.location::geography) AS metres
FROM substations s, (SELECT centroid FROM sites WHERE id = :id) site
WHERE s.spare_capacity_mw > 0
ORDER BY site.centroid <-> s.location
LIMIT 5;

-- Result-cache lookup key (CACHE-01) – rounded lat/long, indexed
CREATE TABLE site_analysis_cache (
    id                uuid PRIMARY KEY,
    lat_rounded       numeric NOT NULL,
    lng_rounded       numeric NOT NULL,
    boundary           geometry(Polygon, 4326),
    vision_refinement jsonb,
    weather_snapshot  jsonb,
    panorama_url       text,
    ml_suitability_score numeric,
    ml_model_version   text,
    created_at         timestamptz NOT NULL DEFAULT now(),
    last_reused_at     timestamptz
);
CREATE UNIQUE INDEX idx_cache_latlng ON site_analysis_cache (lat_rounded, lng_rounded);

CREATE INDEX idx_sites_boundary ON sites USING GIST (boundary);
CREATE INDEX idx_sites_centroid ON sites USING GIST (centroid);

```

Projection note: UTM 44N (`EPSG:32644`) covers 78°–84°E and most of Andhra Pradesh; eastern districts beyond 84°E fall in 45N (`32645`). For a national product, resolve the zone per site or adopt a single equal-area projection. Never buffer on 4326.

15. Build Order

Sprint	Build	Done when
0	PostGIS up; area unit test passing; Terra Draw spike on Google Maps; Sentinel-2 STAC query returning scenes for one tank; repo and CI.	A 100 m square returns $\approx 10,000$ m ² and one real water body loads.
1	Site model, type registry, JSON Schema validation, versioning, bulk import.	Sites of two types can be created, imported and versioned.
2	Area resolver, constraint protocol, capacity resolver, universal pack.	A site with a boundary yields a capacity and a binding constraint.
3	Water index provider, seasonal profile, floating pack, evacuation query.	A tank goes from coordinates to ranked MW with no human input.
4	Generation model, fitness scorer, confidence, output contract, API v1.	Full assessment response for a floating site.
5	Next.js app: site list, map, assessment view, portfolio aggregation, export.	Statewide FSPV inventory demonstrable end to end.
6	Terra Draw capture, rooftop pack, consumption ingest.	Rooftop configuration usable.
7	Survey integration, variance tracking, calibration reporting.	First measured accuracy figure published.
8	Weather API client; result cache (lookup-or-compute, §12.1); vision-refinement provider chained after Solar API/water-index; 3D panorama generation.	A second selection of a previously analysed location returns instantly with no external calls, and the assessment view renders a cropped/refined image plus a 3D panorama.
9	USN capture (manual + bill OCR + payment-proof OCR) wired into the rooftop constraint pack; offline ML scoring baseline trained on data accumulated since Sprint 4, served as additive metadata.	A rooftop site's net-metering/subsidy lookup resolves from a captured USN regardless of input path, and every assessment response carries both the authoritative FIT verdict and a separate ML score.
10 NEW	Extend the Sprint 8 vision-LLM call with structured obstacle detection; auto-apply/reject path for high-confidence	A high-confidence obstacle auto-shrinks a site's usable area and is reversible by an admin; a shaded

obstacles wired into **SITE-05** versioning and **AREA-04** recomputation; extract shading data from the existing Solar API response into the generation derate and fitness sub-score.

roof segment produces a lower generation estimate and a lower fitness score than an identical unshaded one.

16. Testing

Layer	What to assert
Geometry	Known polygon → known area within tolerance. Write this first; it is the cheapest test in the suite and guards every number the product emits.
Projection	Round-trip 4326 → 32644 → 4326 preserves coordinates; buffer of 0.5 m on a 100 m square yields $\approx 9,801 \text{ m}^2$.
Resolver	Property test: recommended is always the minimum; binding always matches; headroom never negative; all-null ceilings → INSUFFICIENT_DATA.
Constraints	Each constraint in isolation, including its insufficient-data path. Gates never enter the minimum.
Water index	Fixture scenes with known water extent; assert $\text{min} < \text{mean} < \text{max}$ and that cloud masking removes flagged pixels.
Scoring	Golden files per verdict, including all-inputs-missing → INSUFFICIENT_DATA, never zero.
Contract	Response schema identical across site types. Snapshot-test the shape.
Reproducibility	Re-running a stored assessment at its recorded pack versions reproduces the original result exactly.
Vision refinement	Mocked vision-API responses: a low-confidence or failed call degrades to <code>insufficient_data</code> without blocking the pipeline; a successful refinement is stored as an annotation and never overwrites the provider boundary.
3D generation	Known boundary + elevation fixture → mesh with expected vertex/area bounds; insufficient elevation data → explicit <code>not_generated</code> , never a fabricated mesh.
ML scoring	Same site, two pipeline runs at the same model version → identical <code>ml_suitability_score</code> ; the score is never read back as, or substituted for, the FIT verdict anywhere in the API response.
Cache	Two site records at the same rounded lat/long → the second creation makes zero external API calls and returns <code>cache_hit=true</code> with a populated <code>reused_from_analysis_id</code> ; different rounding buckets never collide.
USN capture	All three input paths (manual, bill OCR, payment-proof OCR) converge on the same stored <code>usn</code> field with the correct <code>usn_source</code> ; the field is absent from the schema for non-billing-linked site types.

Obstacle detection NEW	Fixture obstacle at/above the confidence threshold → exclusions updated, new SITE-05 version with source=obstacle_detection , AREA-04 recomputed; below threshold → stored advisory-only, exclusions unchanged; admin-reject on an applied obstacle → usable area returns to its prior value with history retained.
Shading NEW	Known Solar API sunshine-hours fixture → expected shading_score ; identical sites differing only in shading produce different generation estimates and different FIT sub-scores; non-Solar-API geometry source → INSUFFICIENT_DATA for the shading sub-component, never a guess.

17. Non-Negotiables

- Area via geography cast or projected CRS. Never planar on 4326.
- No coefficient hard-coded. Every value from a versioned pack, version recorded on every result.
- Results versioned, never overwritten. Any historical assessment reproducible.
- No capacity without its confidence and binding constraint.
- Gates are not zero-value ceilings.
- INSUFFICIENT_DATA is never a low score.
- The resolver contains no site-type conditionals.
- Google imagery: no bulk download, no automated capture, no permanent storage without licence review, no attribution removal, no use as model training data. Sentinel-2 carries no such restriction and is the basis for all automated extraction.
- Every output carries the pre-feasibility limitations statement. The product prioritises; it does not approve.
- **GPT-4 Vision refinement is advisory and stored as an annotation; it never silently overwrites provider geometry.**
- **The ML suitability score is additive metadata; the rules-based FIT verdict remains the sole reproducible, authoritative output.**
- **A cache hit is always recorded as a cache hit — never presented as a fresh computation.**
- **An auto-applied obstacle detection is versioned and reversible exactly like any other boundary change — confidence-gated, never silent, never below the OBS-04 threshold.** NEW